# Week 02

## 입력 · 계산 · 문자열

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

# 지난주 돌아보기

```
birth = "2005"
print(2026 - int(birth))
print(f"{1234500:,}원")
```

- `input()`은 항상 **문자열** → `int()`로 변환
- 오류 메시지는 **마지막 줄부터**
- AI 코드는 **정상 · 경계 · 잘못된 입력**으로 검증

# 오늘의 목표

1. `//` `%` `**`와 우선순위로 생활 문제를 계산한다
2. 실수와 반올림의 함정을 알고 **규칙을 명시해** 계산한다
3. 문자열 슬라이싱과 메서드로 입력을 가공한다
4. 입력 → 처리 → 출력으로 나눈 **여행 경비 계산기**를 만든다

> **오늘의 질문**: 4명이 392,300원을 나눠 내면, 각자 보낸 돈의 합이 정말 총액과 같을까?

# 수업 흐름

| 구간 | 시간 | 활동 |
|---|---|---|
| 도입 · 개념 | 20분 | 연산, 실수의 함정, 문자열 |
| 안내 실습 | 45분 | 노트북 1~2절 |
| 독립 · 짝 실습 | 35분 | 시간 변환기, 여행 경비 계산기 |
| 공유 · 점검 | 20분 | AI 정산 코드 검증 결과 공유 |

## 이번 주 Colab 실습 열기

# 산술 연산자

| 연산자 | 의미 | 예 | 결과 |
|---|---|---|---|
| `/` | 나누기 (항상 실수) | `7 / 2` | `3.5` |
| `//` | 몫 | `7 // 2` | `3` |
| `%` | 나머지 | `7 % 2` | `1` |
| `**` | 거듭제곱 | `2 ** 10` | `1024` |

`6 / 3` → `2.0` (나누어떨어져도 **float**)

# 예상해 보기: 우선순위

```
print(2 + 3 * 4)          # ?
print((2 + 3) * 4)        # ?
print(-2 ** 2)            # ?
print(10 - 4 // 3 * 2)    # ?
```

정답: `14` · `20` · `-4` · `8`

- `**` → `*` `/` `//` `%` → `+` `-`
- 헷갈리면 **괄호**로 의도를 분명하게

# `//`와 `%`: 큰 단위와 작은 단위

```
total_seconds = 7384
hours   = total_seconds // 3600          # 2
minutes = total_seconds % 3600 // 60     # 3
seconds = total_seconds % 60             # 4

q, r = divmod(7384, 60)                  # (123, 4)
```

| 활용 | 식 |
|---|---|
| 짝수 판별 | `n % 2 == 0` |
| 일의 자리 | `n % 10` |
| 십의 자리 | `n // 10 % 10` |

# 실수의 함정 1: 0.1 + 0.2

```
print(0.1 + 0.2)          # 0.30000000000000004
print(0.1 + 0.2 == 0.3)   # False

import math
math.isclose(0.1 + 0.2, 0.3)   # True
```

- 컴퓨터는 실수를 **2진수**로 저장 → 0.1을 정확히 표현 못 함
- 실수끼리 `==` 비교 대신 **`math.isclose`**

# 실수의 함정 2: round(2.5)는?

```
round(2.5), round(3.5), round(4.5)   # (2, 4, 4)
round(2.675, 2)                       # 2.67
```

- Python은 **은행가 반올림**: 가운데 값이면 **짝수** 쪽
- `2.675`는 실제로 `2.67499999...`

> 돈 계산은 `round` 대신 **올림 · 내림 규칙을 명시**

# 올림 · 내림 · 단위 처리

```
import math
math.ceil(33333.4)     # 33334  올림
math.floor(33333.4)    # 33333  내림
int(-2.3)              # -2     0 쪽으로 버림

unit = 100
math.ceil(33333.4 / unit) * unit    # 33400  100원 단위 올림
math.floor(33333.4 / unit) * unit   # 33300  100원 단위 내림
33333 // unit * unit                # 33300  정수일 때
```

**공식**: 단위로 나누고 → 올림/내림 → 다시 곱하기

# 문자열 인덱싱과 슬라이싱

```
문자열:   2  0  2  6  1  2  0  3  4
인덱스:   0  1  2  3  4  5  6  7  8
음수   :  -9 -8 -7 -6 -5 -4 -3 -2 -1
```

```
sid = "202612034"
sid[0], sid[-1]     # '2', '4'
sid[:4]             # '2026'  입학년도
sid[4:6]            # '12'    학과코드
sid[6:]             # '034'   번호
sid[::-1]           # 거꾸로
```

`s[시작:끝]` → 시작 **이상**, 끝 **미만**

# 문자열 메서드

| 메서드 | 예 → 결과 |
|---|---|
| `strip()` | `"  hi "` → `"hi"` |
| `upper()` / `lower()` | `"Py"` → `"PY"` |
| `replace(a, b)` | `"1,000".replace(",", "")` → `"1000"` |
| `split(sep)` | `"a,b".split(",")` → `["a", "b"]` |
| `sep.join(list)` | `"-".join(["a", "b"])` → `"a-b"` |
| `startswith` / `endswith` | `"mokpo.ac.kr".endswith("ac.kr")` → `True` |
| `isdigit()` | `"2026"` → `True`, `"-5"` → `False` |

# 문자열은 바뀌지 않는다

```
raw = "  Mokpo  "
raw.strip()        # 'Mokpo'  ← 새 문자열을 돌려줌
print(raw)         # '  Mokpo  '  원본은 그대로!

raw = raw.strip()  # 결과를 다시 저장해야 바뀜
```

메서드를 **이어서** 쓸 수 있습니다

```
email = "  KimMokpo@Mokpo.AC.kr  "
user, domain = email.strip().lower().split("@")
```

# 금액 문자열 → 숫자

```
price_text = "₩1,250,000"
number = int(price_text.replace("₩", "").replace(",", ""))
print(f"{number * 1.1:,.0f}원")    # 1,375,000원
```

지저분한 입력 정리 순서

1. `strip()` 앞뒤 공백
2. `replace()` 단위 · 쉼표 제거
3. `int()` / `float()` 변환
4. 계산 후 f-string으로 다시 **보기 좋게**

# 실습 1 · 2 (Level 1 → Level 2)

## Level 1 따라하기

- 연산 우선순위, `divmod`, 실수 비교, 반올림 예상
- 학번 슬라이싱, 문자열 메서드

## Level 2 바꾸기

- 시간 변환기: 값 3개 바꾸기 + `f"{h:02d}"`로 두 자리 출력
- 이메일 정리: 공백 제거 → 소문자 → 아이디/도메인 분리

# 실습 3 (Level 3): 여행 경비 계산기

**입력 → 처리 → 출력** 분리

```
expense_text = "숙박:120,000 / 교통:64,800 / 식비:183,500 / 입장료:24,000"
people_text = "4"
```

1. `/`로 항목을 나누고 `:`로 이름과 금액 분리
2. 쉼표 제거 후 `int` → 총액 누적
3. 1인당 금액 = **100원 단위 올림**
4. 영수증 출력

> 금액 안에 쉼표가 있어서 항목 구분자는 `/` — **구분자 선택도 설계**

# 영수증 출력과 검증

```
print(f"{name:<8}{amount:>14,}원")
```

```
숙박            120,000원
교통             64,800원
...
1인당(올림)    98,100원
```

```
collected = per_person * people
assert collected >= total                  # 부족하면 안 됨
assert collected - total < people * 100    # 남는 돈은 인원×100 미만
```

# AI가 준 정산 코드

```python
def split_bill(total, people):
    each = round(total / people, -2)   # 100원 단위 반올림
    return each
```

| 총액 / 인원 | 1인 금액 | 모은 돈 – 총액 |
|---|---|---|
| 100,000 / 4 | 25,000.0 | 0 |
| 100,000 / 3 | 33,300.0 | **−100** |
| 50,000 / 6 | 8,300.0 | **−200** |

결과가 **실수**이고, 모은 돈이 **부족**할 수 있다

# 왜 이런 코드가 나왔을까?

- 프롬프트: "총액을 n명이 나눠 내는 정산 코드"
- **규칙이 없었다**: 부족해도 되는지, 단위는?
- **검증 기준이 없었다**: 합계가 총액과 같아야 하는지?

> AI는 **말하지 않은 규칙**을 지켜 주지 않습니다

# 규칙을 먼저 정하고 코드로

```
def split_bill_exact(total, people, unit=100):
    """n-1명은 unit 단위 내림, 마지막 사람이 나머지."""
    each = total // people // unit * unit
    last = total - each * (people - 1)
    return each, last

split_bill_exact(100000, 3)    # (33300, 33400)
```

**불변 조건**으로 검증

```
assert each * (people - 1) + last == total
```

# 좋은 프롬프트로 다시 묻기

```
총액(정수, 원)과 인원(정수)을 받아 정산 금액을 계산하는 함수.
- 금액은 정수, 100원 단위로 처리
- n-1명은 100원 단위로 내림한 같은 금액, 마지막 사람이 나머지
- '일반 금액 × (n-1) + 마지막 금액 == 총액'이 반드시 성립
- 이 조건을 확인하는 assert 테스트 5개 이상 포함
```

**규칙 + 검증 조건**을 넣으면 → 답이 정확해지고, 내가 확인할 기준도 생긴다

# 흔한 실수 모음

| 실수 | 결과 | 해결 |
|---|---|---|
| `int("1,000")` | `ValueError` | `replace(",", "")` 먼저 |
| `s.strip()`만 하고 저장 안 함 | 원본 그대로 | `s = s.strip()` |
| `price == 0.3` 실수 비교 | 엉뚱한 `False` | `math.isclose` |
| 돈 계산에 `round` | 합계 불일치 | `ceil` / `floor` + 나머지 처리 |
| `sid[4:5]`로 두 글자 기대 | 한 글자 | 끝은 **미만** |

# 이번 주 과제 (10점)

| 문제 | 내용 | 배점 |
|---|---|---|
| 1 | `to_hms(sec)` → `"02:03:04"` | 2 |
| 2 | `parse_student_id(sid)` 학번 분석 | 2 |
| 3 | `make_receipt(order_text)` 영수증 문자열 | 3 |
| 4 | `split_exact(total, people, unit)` 정확한 정산 (AI 협업) | 3 |

- 문제 4는 **규칙과 검증 조건을 담은 프롬프트**를 기록
- 자동 확인 ✅ + AI 활용 기록 + 성찰

# 자기 점검

1. `7384`초를 시·분·초로 나누는 식은?
2. `round(2.5)`가 2인 이유는?
3. `"₩1,250,000"`을 정수로 바꾸는 순서는?
4. `"202612034"[4:6]`의 결과는?
5. 정산 결과를 검증하는 **불변 조건**은?

# 정리와 다음 주

| 오늘 | 핵심 |
|---|---|
| 연산 | `//` `%` `divmod`, 괄호로 우선순위 명확히 |
| 실수 | `isclose`, 은행가 반올림, 올림·내림 명시 |
| 문자열 | 슬라이싱, `strip` `split` `replace` `join` |
| 설계 | 입력 → 처리 → 출력, 구분자 선택 |
| AI | 규칙 + 검증 조건을 프롬프트에, 불변 조건으로 확인 |

**다음 주 · Week 03** 조건문과 반복문 — 장학 기준 판정기와 경계값 검증